

MathViz 3D — Frontend Layer Build Guide

Scope: This document covers the design and implementation guidance for the React + TypeScript browser client — the 3D Visualization Viewport with HUD overlay, the Step Timeline Bar, and the LLM Assistant Panel. It covers component architecture, state management, WebSocket handling, Three.js rendering, and the KaTeX equation overlay system.

1. Purpose and Role

The frontend is the user-facing surface of MathViz 3D. It is a single-page application split into three independent UI regions that share state through a centralized Zustand store:

- **3D Visualization Viewport** — renders geometry using Three.js WebGL 2, overlays the equation HUD via CSS2DRenderer, and drives step transition animations.
- **Step Timeline Bar** — playback controls (play/pause, step navigation, speed, scrubber) and step marker tooltips.
- **LLM Assistant Panel** — chat interface with LaTeX rendering, streaming partial responses, and step chips that link to viewport steps.

The frontend has no business logic beyond UI orchestration. It does not interpret geometry data, does not call the LLM directly, and does not manage session authentication. All of that is handled server-side.

2. Project Structure

```
frontend/
├── index.html
├── vite.config.ts
├── tsconfig.json
├── package.json
├──
├── src/
│   ├── main.tsx           # React root, app entry point
│   ├── App.tsx            # Top-level layout: Viewport + Timeline + Panel
│   ├──
│   ├── store/
│   │   ├── index.ts       # Zustand store composition
│   │   ├── sceneSlice.ts  # layers[], activeLayers[], stepIndex, totalSteps
│   │   └── playbackSlice.ts # isPlaying, speed, dwellMs, transitionMs
```

- └─ hudSlice.ts # activeEquation, domain, variables, narrationText
- └─ chatSlice.ts # messages[], pendingStepBundle, streamingText
- └─ sessionSlice.ts # sessionId, userId, sceneId, wsUrl, preferences
-
- └─ viewport/
 - └─ ViewportContainer.tsx # Mounts the Three.js canvas, handles resize
 - └─ SceneManager.ts # Three.js scene, camera, renderer lifecycle
 - └─ LayerManager.ts # Adds/removes/updates Three.js mesh objects per layer
 - └─ TransitionController.ts # Animates step transitions (fade, grow, draw, etc.)
 - └─ HUDOverlay.tsx # React component: equation panel, domain inspector
 - └─ EquationLabel.ts # CSS2DObject factory for 3D floating labels
 - └─ ContextMenu.tsx # Right-click geometry inspector panel
 - └─ hooks/
 - └─ useThreeSetup.ts # Initialize renderer, scene, camera
 - └─ useAnimationLoop.ts # requestAnimationFrame loop management
 - └─ useStepBundleApply.ts # Apply incoming StepBundle to the Three.js scene
 - └─ useResizeObserver.ts # Canvas resize handling
- └─ timeline/
 - └─ TimelineBar.tsx # Root timeline component
 - └─ StepMarker.tsx # Individual step marker with tooltip
 - └─ Scrubber.tsx # Drag-to-seek control
 - └─ PlaybackControls.tsx # Play/pause, forward, back, speed selector, reset
 - └─ hooks/
 - └─ usePlaybackTimer.ts # Auto-advance timer at configured dwell time
- └─ panel/
 - └─ AssistantPanel.tsx # Root chat panel component
 - └─ MessageList.tsx # Scrollable message history
 - └─ MessageBubble.tsx # Single message with LaTeX and step chips
 - └─ StepChip.tsx # Clickable chip linking to a viewport step
 - └─ InputBar.tsx # Text input with send button
 - └─ StreamingIndicator.tsx # Animated indicator during LLM streaming
 - └─ hooks/
 - └─ useSSEStream.ts # Consumes SSE stream from LLM Service
- └─ websocket/
 - └─ WebSocketClient.ts # WebSocket connection, reconnect, binary frame handling
 - └─ BundleDecoder.ts # msgpack decoding of incoming StepBundle frames
 - └─ PlaybackEventSender.ts # Sends playback events (JSON) over the WebSocket
-
- └─ geometry/
 - └─ BufferBuilder.ts # Converts LayerPayload arrays → Three.js BufferGeometry
 - └─ MaterialFactory.ts # Creates Three.js materials from render hints

```
| | └─ InstancedArrows.ts      # Builds InstancedMesh for vector field arrows
| |
| └─ math/
|   └─ katex.ts              # KaTeX render helper (string → HTML string)
|   └─ latex.ts              # LaTeX utilities: sanitize, truncate for display
|
| └─ api/
|   └─ chatApi.ts            # POST /chat to LLM Service
|   └─ sceneApi.ts           # GET /scenes/{id}/steps/{n}, GET /sessions/{id}/history
|
| └─ types/
|   └─ StepBundle.ts          # TypeScript types mirroring the wire format
|   └─ SceneDescription.ts     # TypeScript types for SceneDescription (read-only reference)
|   └─ PlaybackEvent.ts       # Playback event union type
|
└─ public/
  └─ fonts/                  # KaTeX font files (self-hosted)
  └─ tests/
    └─ unit/
      └─ BufferBuilder.test.ts
      └─ BundleDecoder.test.ts
      └─ store.test.ts
    └─ e2e/
      └─ playwright/
        └─ viewport.spec.ts
        └─ timeline.spec.ts
        └─ hud.spec.ts
```

3. Technology Stack

Concern	Technology	Rationale
Framework	React 18 + TypeScript	Concurrent rendering; Suspense for step streaming
Build tool	Vite 5	Sub-second HMR; native ES modules
3D rendering	Three.js r152+ (WebGL 2)	Mature; instanced geometry; shader support
HUD equation overlay	CSS2DRenderer (Three.js addon)	Renders DOM elements in 3D space, always facing camera

Concern	Technology	Rationale
Math label rendering	KaTeX	10× faster than MathJax; critical for real-time HUD updates
State management	Zustand	Minimal boilerplate; slice-based; no provider wrapping
WebSocket transport	Native WebSocket + msgpack-lite	Binary frames; 30–50% smaller than JSON for geometry
SSE (LLM streaming)	Native <code>EventSource</code>	Text streaming from LLM Service
Styling	CSS Modules + CSS custom properties	Scoped styles; theme tokens for dark mode
Testing (unit)	Vitest + Testing Library	Fast; native ESM; same config as Vite
Testing (e2e)	Playwright	Screenshot comparison for visual regression

4. State Management (`store/`)

All cross-component communication goes through the Zustand store. No prop drilling. No direct cross-component calls. Components subscribe to only the slices they need.

4.1 `sceneSlice.ts`

Owns the current visualization state.

```
typescript
```

```

interface SceneSlice {
  // All layers ever sent in this scene session (accumulated)
  layers: LayerPayload[];
  // Subset currently visible (respects layer_mode and user visibility toggles)
  activeLayers: LayerPayload[];
  // Current step (1-based)
  stepIndex: number;
  totalSteps: number;
  // Map of layerId → visibility (user can toggle)
  layerVisibility: Record<string, boolean>;
  // Actions
  applyStepBundle: (bundle: StepBundle) => void;
  toggleLayerVisibility: (layerId: string) => void;
  jumpToStep: (step: number, bundleHistory: StepBundle[]) => void;
  reset: () => void;
}

```

`applyStepBundle` is the central reducer. It inspects `bundle.layer_mode`:

- `add`: append new layers from the bundle to `layers`; set `activeLayers` to all layers with `is_visible = true`.
- `replace`: replace `layers` entirely with the bundle's layers; reset `layerVisibility`.
- `highlight`: keep all existing layers but set their opacity to a dimmed value in `activeLayers`; add new layers at full opacity.

4.2 `playbackSlice.ts`

Owns playback state. Consumers: `TimelineBar`, `ViewportContainer` (for transition timing).

```

typescript

interface PlaybackSlice {
  isPlaying: boolean;
  speed: 0.5 | 1 | 2 | 4;
  dwellMs: number;    // base dwell time (from server hint, default 4000)
  transitionMs: number; // transition animation duration, default 400
  setPlaying: (playing: boolean) => void;
  setSpeed: (speed: number) => void;
}

```

`dwellMs` is derived: `baseDwellMs / speed`. Components that need the effective dwell time read this derived value, not `speed` directly.

4.3 `hudSlice.ts`

Owns HUD overlay content. Updated synchronously on every `applyStepBundle` call.

```
typescript

interface HUDSlice {
  activeEquation: string; // LaTeX string for the Active Equation Panel
  stepLabel: string;      // e.g. "Step 3/7: Add n=3 harmonic"
  domain: DomainDisplay;  // { axes: { name, min, max, steps }[] }
  variables: VariableDisplay[]; // { name, value }[]
  narrationText: string;
  setFromBundle: (bundle: StepBundle) => void;
}
```

`setFromBundle` extracts `hud_equation`, `step_label`, `narration`, and domain info from the bundle and updates all fields atomically. This avoids partial renders where the equation updates but the narration hasn't yet.

4.4 `chatSlice.ts`

Owns the LLM chat state.

```
typescript

interface ChatSlice {
  messages: ChatMessage[];
  streamingText: string; // accumulated partial text during SSE stream
  isStreaming: boolean;
  pendingSceneId: string | null;
  appendUserMessage: (text: string) => void;
  appendStreamingDelta: (delta: string) => void;
  finalizeAssistantMessage: (sceneId: string | null) => void;
  addStepChips: (sceneId: string, totalSteps: number) => void;
}
```

`ChatMessage` union type: `UserMessage` | `AssistantMessage` | `SystemMessage`. `AssistantMessage` includes an optional `stepChips: StepChip[]` array populated when `scene_ready` SSE event arrives.

4.5 `sessionSlice.ts`

Owns session identity and WebSocket connection metadata.

```
typescript
```

```
interface SessionSlice {
  sessionId: string;
  userId: string;
  sceneId: string | null;
  wsUrl: string | null;
  wsStatus: 'disconnected' | 'connecting' | 'connected' | 'error';
  preferences: UserPreferences;
  setScene: (sceneId: string, wsUrl: string) => void;
  setWsStatus: (status: WsStatus) => void;
}
```

5. Viewport Layer (`viewport/`)

5.1 ViewportContainer (`ViewportContainer.tsx`)

The React component that owns the `<canvas>` element. Responsibilities:

- Mount and unmount the Three.js renderer to the canvas ref.
- Observe canvas resize events and update the renderer and camera aspect ratio.
- Render the `HUDOverlay` as a React component overlaid on the canvas using `position: absolute`.
- Render the `ContextMenu` component conditionally on right-click.
- Connect `useStepBundleApply` to the Zustand `sceneSlice` to drive Three.js updates from state changes.

This component itself has no Three.js logic — it delegates entirely to `SceneManager` and hooks. Its job is to bridge the React tree and the imperative Three.js world.

5.2 SceneManager (`SceneManager.ts`)

A plain TypeScript class (not a React component) that owns the Three.js scene graph. Instantiated once and stored in a React ref inside `ViewportContainer`.

Responsibilities:

- Create and configure the `WebGLRenderer` (WebGL 2 context, tone mapping, shadow maps off for performance).
- Create the `PerspectiveCamera` with configurable FOV.
- Create the `CSS2DRenderer` for equation labels — rendered as a transparent DOM layer over the canvas.
- Set up `OrbitControls` for mouse/touch camera navigation.
- Add ambient and directional lights with sensible defaults.

- Expose `addLayer(payload: LayerPayload)`, `removeLayer(layerId: string)`, `dimLayer(layerId: string, opacity: number)`, and `clearAll()` methods called by `LayerManager`.
- Run the render loop via `useAnimationLoop`.

The `SceneManager` never reads from Zustand directly — it receives instructions via method calls from hooks and `LayerManager`.

5.3 LayerManager (`LayerManager.ts`)

Translates `LayerPayload` objects (from decoded WebSocket bundles) into Three.js objects and manages their lifecycle in the scene.

`addLayer(payload: LayerPayload, scene: THREE.Scene): void`

1. Call `BufferBuilder.build(payload)` to get a `THREE.BufferGeometry`.
2. Call `MaterialFactory.create(payload)` to get a `THREE.Material`.
3. Decide the Three.js object type:
 - Surfaces: `THREE.Mesh`
 - Curves/line strips: `THREE.Line` or `THREE.LineSegments`
 - Vector field arrows: `THREE.InstancedMesh` via `InstancedArrows`
4. Assign `userData.layerId = payload.layer_id` and `userData.sourceExpression = payload.source_expression` for right-click inspection.
5. Add to the scene. Store a reference in a `Map<layerId, THREE.Object3D>` for later removal.
6. If the payload includes annotations, create `CSS2DObject` labels and attach them to the mesh or to fixed world positions.

`removeLayer(layerId: string, scene: THREE.Scene): void` Look up the object by `layerId`, remove from scene, dispose geometry and material to free GPU memory.

`dimLayer(layerId: string, opacity: number): void` Look up the object, set `material.opacity = opacity` and `material.transparent = true`.

`clearAll(scene: THREE.Scene): void` Remove and dispose all tracked objects.

5.4 TransitionController (`TransitionController.ts`)

Drives entry animations for new layers. Called immediately after `LayerManager.addLayer` completes.

Each transition type is an async function that runs an animation over `transitionMs` milliseconds using a `requestAnimationFrame`-based timer. The animation updates Three.js object properties each frame.

fade_in: Set `material.opacity = 0`, `material.transparent = true`. Animate opacity from 0 $\rightarrow$ target opacity over `transitionMs`.

grow: Set `object.scale.set(0, 0, 0)`. Animate scale from (0,0,0) $\rightarrow$ (1,1,1) using an ease-out cubic curve.

draw: Requires the arc-length array from the bundle. Use `THREE.Line` with a `DrawRange`. Animate `geometry.setDrawRange(0, n)` where n increases from 0 to the total vertex count, paced by arc-length so the drawing speed is uniform along the curve.

transform: Requires two geometry buffers (before and after). Use `THREE.Mesh` morph targets: add the "before" positions as `morphAttributes.position[0]` and the "after" as the base position. Animate `mesh.morphTargetInfluences[0]` from 1 $\rightarrow$ 0 over `transitionMs`.

highlight: Dim all existing layers to 0.3 opacity, then fade in the new layer at full opacity. Orchestrates calls to `LayerManager.dimLayer` followed by a `fade_in` on the new layer.

Transitions can be cancelled (e.g., user clicks `StepBack` mid-transition). `TransitionController` tracks the active animation frame ID per layer and cancels it on request.

5.5 HUD Overlay (`HUDOverlay.tsx`)

A React component rendered on top of the canvas using absolute positioning. Reads from `hudSlice` and `sceneSlice`. Renders four sub-components:

ActiveEquationPanel (top-left):

```
tsx

// Renders: "Step 3/7: Add n=3 harmonic"
// Below it: KaTeX-rendered hud_equation
<div className={styles.equationPanel}>
  <span className={styles.stepLabel}>{stepLabel}</span>
  <div
    className={styles.katexContainer}
    dangerouslySetInnerHTML={{ __html: katex.renderToString(activeEquation) }}
  />
</div>
```

DomainInspector (bottom-left): Lists each domain axis with its min, max, and step count. Lists all active variables and their current values. Updates on every step transition.

LayerList (right sidebar, collapsible): For each layer in `activeLayers`, shows: a color swatch (matching the layer's `color_hint`), the layer name (from `step_label`), a KaTeX-rendered `source_expression`, and a visibility toggle checkbox linked to `sceneSlice.toggleLayerVisibility`. Collapsed by default on mobile.

NarrationTicker (bottom strip): Displays `narrationText` from `hudSlice`. Uses a CSS fade-in animation triggered by a key change on each new narration string. Text wraps to two lines max; longer narrations are truncated with an expand button.

StepProgressIndicator (bottom-centre): Renders `totalSteps` numbered markers. Current step is highlighted. Completed steps show a checkmark. Future steps are greyed. Each marker is clickable and dispatches a `JumpTo` playback event.

5.6 Floating Equation Labels (`EquationLabel.ts`)

Factory function: `createEquationLabel(latex: string, position: THREE.Vector3, style: AnnotationStyle): CSS2DObject`

1. Create a DOM `div` element.
2. Set `innerHTML` to `katex.renderToString(latex, { throwOnError: false })`.
3. Apply CSS class based on `style` (default | highlight | dim).
4. Wrap in `CSS2DObject` and set its position.
5. Return the object for the caller to add to the scene.

Labels automatically face the camera (`CSS2DRenderer` handles this). They scale with zoom: use a CSS `transform: scale()` driven by a `zoom` uniform updated in the animation loop.

5.7 Context Menu (`ContextMenu.tsx`)

On right-click on the canvas, perform a Three.js raycasting operation to find the clicked object. If it has `userData.layerId`, display a context panel showing:

- Source expression (LaTeX rendered via KaTeX)
- Concept type (human-readable label)
- Evaluation domain (formatted from the step's domain spec)
- Computation time (from bundle metadata if available)
- Step it belongs to

The context menu is a React portal rendered at the click position. It dismisses on outside click or Escape key.

6. Geometry Decoding (`geometry/`)

6.1 BufferBuilder (`BufferBuilder.ts`)

Entry point: `build(payload: LayerPayload): THREE.BufferGeometry`

1. Decode `payload.vertex_buffer` (msgpack binary blob) to a `Float32Array`.

2. Decode `payload.index_buffer` to a `Uint32Array`.
3. Decode `payload.normal_buffer` to a `Float32Array` (may be empty for curves).
4. Decode `payload.uv_buffer` to a `Float32Array` (may be empty).
5. Create a `THREE.BufferGeometry`.
6. Set `geometry.setAttribute('position', new THREE.BufferAttribute(vertices, 3))`.
7. If normals non-empty: `geometry.setAttribute('normal', new THREE.BufferAttribute(normals, 3))`.
8. If UVs non-empty: `geometry.setAttribute('uv', new THREE.BufferAttribute(uvs, 2))`.
9. `geometry.setIndex(new THREE.BufferAttribute(indices, 1))`.
10. If normals are empty and it is a surface: call `geometry.computeVertexNormals()` as fallback.
11. Call `geometry.computeBoundingBox()` and `geometry.computeBoundingSphere()` for frustum culling.

For line-strip geometry (curves): use a non-indexed geometry. The index buffer encodes segment breaks as `Uint32.MAX` — split into separate `LineSegments` objects at these positions.

6.2 MaterialFactory (`MaterialFactory.ts`)

Entry point: `create(payload: LayerPayload, renderHints: RenderHints): THREE.Material`

Decision tree based on `payload.concept_type`:

- **Surfaces** (`function_3d`, `parametric_surface`, `implicit_surface`): `THREE.MeshStandardMaterial` with `color`, `opacity`, `roughness: 0.6`, `metalness: 0.1`. Enable `side: THREE.DoubleSide` for implicit surfaces (no winding order guarantee). Enable `transparent: true` if `opacity < 1`.
- **Curves** (`function_2d`, `parametric_curve_*`, `derivative_tangent`): `THREE.LineBasicMaterial` with `color`, `linewidth` (note: WebGL 2 ignores `linewidth > 1` on most GPUs — use `THREE.Line2` from Three.js addons with `LineMaterial` for thick lines).
- **Vector field arrows** (instanced): `THREE.MeshStandardMaterial` on the `InstancedMesh`. Color is set per-instance via `instanceColor` attribute from the instance buffer.
- **Riemann rectangles**: `THREE.MeshStandardMaterial` with semi-transparent fill and a `THREE.LineSegments` outline using `THREE.EdgesGeometry`.

Parse `payload.color_hint` (hex string) to a `THREE.Color`. If `render_hints.show_wireframe` is true, set `wireframe: true` on the material.

6.3 InstancedArrows (`InstancedArrows.ts`)

Builds the instanced mesh for vector field arrow geometry.

1. Create a unit arrow base mesh: a `CylinderGeometry` for the shaft and a `ConeGeometry` for the head, merged into a single `BufferGeometry` using `THREE.BufferGeometryUtils.mergeGeometries`.

2. Create an `InstancedMesh` with `count = number of arrows`.
 3. Decode `payload.instance_buffer`: interleaved `[tx, ty, tz, qx, qy, qz, qw, scale]` per instance.
 4. For each instance: set the transform matrix from translation + quaternion + uniform scale using `THREE.Matrix4.compose`. Apply via `instancedMesh.setMatrixAt(i, matrix)`.
 5. If color varies per instance: set `instancedMesh.setColorAt(i, color)` from per-instance color data.
 6. Call `instancedMesh.instanceMatrix.needsUpdate = true`.
-

7. WebSocket Layer (`websocket/`)

7.1 WebSocketClient (`WebSocketClient.ts`)

A plain TypeScript class managing the WebSocket lifecycle.

Connection: `connect(wsUrl: string, token: string): void` Opens `new WebSocket(wsUrl + '?token=' + token)`. Updates `sessionSlice.wsStatus` on `onopen`, `onclose`, `onerror`.

Binary frame handling: `onmessage` receives binary frames (`event.data` is `ArrayBuffer`). Pass to `BundleDecoder.decode()` and dispatch the resulting `StepBundle` to `sceneSlice.applyStepBundle` and `hudSlice.setFromBundle`.

Reconnection: On `onclose` (unless intentional disconnect), wait 1 s and retry `connect`. After 3 failed attempts, set `wsStatus = 'error'` and show the user a reconnection prompt. On successful reconnect, request the current step bundle via `GET /scenes/{id}/steps/{currentStep}` to resync state.

Control messages: `onmessage` also handles JSON text frames. Currently used for:

- `{"type": "buffer", "step": N}` — server is still computing; show a loading indicator for step N.
- `{"type": "error", "message": str}` — scene-level error; display in the chat panel.

7.2 BundleDecoder (`BundleDecoder.ts`)

Entry point: `decode(data: ArrayBuffer): StepBundle`

1. Check the first byte — the version byte. Reject if not the expected version.
2. Pass the remaining bytes to `msgpack.decode()` (using `msgpack-lite` or `@msgpack/msgpack`).
3. Validate the decoded object against the `StepBundle` TypeScript type (runtime check via a lightweight type guard, not full Zod validation — speed matters here).
4. Return the typed `StepBundle`.

Binary array fields (`vertex_buffer`, `normal_buffer`, `index_buffer`, `uv_buffer`, `instance_buffer`) arrive as `Uint8Array` from msgpack. Store them as-is on the `LayerPayload` — `BufferBuilder` will view them as the correct typed arrays.

7.3 PlaybackEventSender (`PlaybackEventSender.ts`)

Sends playback events to the server over the same WebSocket as a JSON text frame.

typescript

```
class PlaybackEventSender {
  send(event: PlaybackEvent): void {
    if (this.ws.readyState !== WebSocket.OPEN) return; // drop silently if not connected
    this.ws.send(JSON.stringify(event));
  }
}
```

`PlaybackEvent` union type:

typescript

```
type PlaybackEvent =
  | { type: 'step_forward' }
  | { type: 'step_back' }
  | { type: 'jump'; step: number }
  | { type: 'reset' }
  | { type: 'set_speed'; multiplier: 0.5 | 1 | 2 | 4 }
  | { type: 'set_playing'; playing: boolean }
```

8. Step Timeline Bar (`timeline/`)

8.1 TimelineBar (`TimelineBar.tsx`)

Root component. A fixed horizontal strip below the viewport. Reads from `sceneSlice` and `playbackSlice`. Renders `PlaybackControls` and the step marker row.

On mount, registers keyboard shortcuts:

- `Space` → toggle play/pause
- `ArrowRight` → step forward
- `ArrowLeft` → step back
- `Home` → reset to step 1

- `End` → jump to last step

Keyboard events are handled at the document level with `useEffect`, not on the component — this ensures they work regardless of focus state.

8.2 StepMarker (`StepMarker.tsx`)

One circular marker per step. Props: `stepIndex`, `isActive`, `isCompleted`, `label`, `equation` (LaTeX), `narration`.

Visual states:

- **Completed** (step < current): filled circle with checkmark, full opacity.
- **Active** (step === current): filled circle with ring highlight, larger size.
- **Future** (step > current): empty circle, reduced opacity.

On hover: shows a tooltip containing the `label`, a KaTeX-rendered `equation`, and the first sentence of `narration`.

On click: dispatch `JumpTo(stepIndex)` via `PlaybackEventSender` and update `sceneSlice` via `jumpToStep`.

8.3 Scrubber (`Scrubber.tsx`)

A drag-to-seeek control rendered as a horizontal track with a draggable thumb. Position maps linearly to step index.

Dragging behavior: on `pointerdown`, capture the pointer. On `pointermove`, calculate the new step index from the drag position. Update `sceneSlice.stepIndex` immediately (optimistic update — the bundle is already in cache). On `pointerup`, release. Dispatch `JumpTo` via `PlaybackEventSender` on pointer up (not on every move event — avoid spamming the server).

The scrubber thumb position is derived from `sceneSlice.stepIndex / totalSteps`. It is not controlled by local state.

8.4 PlaybackControls (`PlaybackControls.tsx`)

Renders: play/pause button, step-back button, step-forward button, speed selector (dropdown: 0.5×, 1×, 2×, 4×), reset button.

All buttons dispatch to both the Zustand store (for immediate local feedback) and `PlaybackEventSender` (to sync with the server sequencer). This dual-dispatch pattern ensures the UI stays responsive even if the WebSocket has latency.

8.5 `usePlaybackTimer` hook

```
function usePlaybackTimer(onStepAdvance: () => void): void {
  const { isPlaying, dwellMs } = usePlaybackStore();
  useEffect(() => {
    if (!isPlaying) return;
    const id = setInterval(onStepAdvance, dwellMs);
    return () => clearInterval(id);
  }, [isPlaying, dwellMs]);
}
```

`onStepAdvance` dispatches a `StepForward` event. The interval resets whenever `dwellMs` or `isPlaying` changes (speed changes take effect on the next interval boundary).

9. LLM Assistant Panel (`panel/`)

9.1 AssistantPanel (`AssistantPanel.tsx`)

Root panel component. A vertically scrolling panel fixed to the right side of the screen (or bottom on mobile). Contains `MessageList` above and `InputBar` below.

On mount: restores conversation history via `GET /sessions/{sessionId}/history` and populates `chatSlice`.

9.2 MessageList (`MessageList.tsx`)

Renders the list of `ChatMessage` objects from `chatSlice`. Auto-scrolls to the bottom on each new message using a scroll anchor div and `scrollIntoView`.

During streaming: renders `streamingText` as a partial message at the bottom, below all completed messages, with a blinking cursor CSS animation.

9.3 MessageBubble (`MessageBubble.tsx`)

Renders a single message. For assistant messages: parses the content string for LaTeX delimiters (`($...$)` for inline, `($$...$$)` for block) and renders them via KaTeX. Non-LaTeX segments are plain text.

Below the message content, renders `StepChip` components if the message has `stepChips`.

LaTeX rendering: split the content string on LaTeX delimiters using a regex. Map each segment: LaTeX segments → `dangerouslySetInnerHTML` with KaTeX output; plain text segments → escaped text nodes. This mixed rendering is the most complex part of the panel — test it thoroughly with edge cases (nested delimiters, escaped dollar signs, multiline block LaTeX).

9.4 StepChip (`StepChip.tsx`)

A small clickable pill button: "Step N: {step_label}". On click:

1. Dispatch `JumpTo(N)` to `PlaybackEventSender`.
2. Call `sceneSlice.jumpToStep(N, bundleHistory)`.
3. Optionally scroll the viewport into view (if the viewport is offscreen on mobile).

Chips are highlighted if their step is the current `sceneSlice.stepIndex`.

9.5 InputBar (`InputBar.tsx`)

A `<textarea>` (auto-resizing, max 4 lines) with a send button. On send:

1. Append the user message to `chatSlice`.
2. Clear the input.
3. Call `useSSEStream` to POST to the LLM Service and consume the SSE stream.

The send button is disabled while `chatSlice.isStreaming` is true.

9.6 `useSSEStream` hook

typescript

```
function useSSEStream(message: string): void {
  // POST /chat to LLM Service with fetch() using streaming response
  // Read the response body as a ReadableStream
  // Parse SSE events: text_delta → chatSlice.appendStreamingDelta
  //      scene_ready → sessionSlice.setScene + chatSlice.finalizeAssistantMessage
  //      done → chatSlice.finalizeAssistantMessage
  // On scene_ready: WebSocketClient.connect(wsUrl)
}
```

Use `fetch()` with `{ method: 'POST', body: ... }` and read `response.body` as a `ReadableStream<Uint8Array>`. Decode lines with a `TextDecoder`. Parse SSE event lines manually (`event:` and `data:` prefixes). This gives more control than `EventSource`, which does not support POST.

On `scene_ready`: connect the `WebSocketClient` to the new `wsUrl`. The WebSocket connection starts buffering step bundles immediately — Step 1 may arrive before the user even finishes reading the assistant's response.

10. Rendering Performance

10.1 Animation Loop (`useAnimationLoop.ts`)

typescript


```
function useAnimationLoop(sceneManager: SceneManager): void {
  useEffect(() => {
    let id: number;
    const loop = () => {
      id = requestAnimationFrame(loop);
      sceneManager.render(); // renders both WebGLRenderer and CSS2DRenderer
    };
    loop();
    return () => cancelAnimationFrame(id);
  }, [sceneManager]);
}
```

The loop runs continuously at the display refresh rate. `sceneManager.render()` calls both `renderer.render(scene, camera)` and `css2dRenderer.render(scene, camera)` on every frame. This is intentional — the CSS2DRenderer must re-render to keep equation labels correctly projected as the camera moves.

10.2 Geometry Disposal

Every time a layer is removed (`LayerManager.removeLayer`), both the geometry and material are disposed:

```
typescript
geometry.dispose();
material.dispose();
if (material.map) material.map.dispose();
```

This prevents GPU memory leaks during long sessions with many step transitions. The `LayerManager` tracks all created objects in a `Map` for this purpose.

10.3 Draw Call Budget

Keep draw calls under 100 per frame for smooth 60 fps on mid-range hardware.

Strategies:

- Vector field arrows use `InstancedMesh` — all arrows in one draw call regardless of count.
- Merge multiple curve segments where possible using `THREE.BufferGeometryUtils.mergeGeometries` before adding to the scene.
- Use `mesh.frustumCulled = true` (default) — Three.js skips draw calls for off-screen geometry automatically.
- For scenes with many accumulated layers (12-step sequences with `add` mode), consider merging completed non-animated layers into a single static mesh once their transition is done.

11. Responsive Design and Accessibility

11.1 Layout

Three layout modes based on viewport width (CSS custom properties drive breakpoints):

Desktop ($\geq 1200\text{px}$): Viewport takes the left 70% of the screen. Assistant panel is a fixed-width right column. Timeline bar is below the viewport only.

Tablet (768–1199px): Viewport is full-width. Timeline bar is below. Assistant panel slides in as a drawer from the right, triggered by a chat button.

Mobile ($< 768\text{px}$): Viewport is full-width, top half. Timeline bar below the viewport. Assistant panel is a bottom sheet.

The HUD overlay adapts: on mobile, the Domain Inspector and Layer List are collapsed by default. The Active Equation Panel is shown in a smaller font.

11.2 Accessibility

- All interactive elements (step markers, playback controls, layer list toggles) have `aria-label` attributes.
- Step markers have `role="button"` and `aria-pressed` (for the active step).
- KaTeX-rendered equations have `aria-label` set to a plain-text transcription of the LaTeX (e.g., `aria-label="S sub 3 equals sin x plus sin 3x divided by 3"`).
- The Narration Ticker text is also announced via an `aria-live="polite"` region so screen readers announce each step's narration.
- The canvas itself has `aria-label="3D mathematical visualization"` and `role="img"`.
- Keyboard navigation: all interactive controls are reachable via Tab. Step markers are a `role="listbox"` with arrow key navigation within the marker row.

12. Testing Strategy

12.1 Unit Tests (Vitest)

`BufferBuilder.test.ts`:

- Given a known `LayerPayload` with a simple grid (2×2 quad), assert the resulting `BufferGeometry` has the correct vertex count, index count, and bounding box.
- Given an empty normal buffer, assert `computeVertexNormals` is called (mock Three.js).

- Given a line-strip payload with a `UInt32.MAX` sentinel, assert the geometry is split at the correct position.

`BundleDecoder.test.ts`:

- Given a valid msgpack binary fixture, assert the decoded `StepBundle` has all expected fields.
- Given a wrong version byte, assert an error is thrown.
- Given a truncated binary, assert graceful failure (no crash).

`store.test.ts`:

- `applyStepBundle` with `layer_mode = 'add'` appends layers correctly.
- `applyStepBundle` with `layer_mode = 'replace'` replaces layers correctly.
- `jumpToStep` reconstructs `activeLayers` correctly for a multi-step `add` sequence.
- `toggleLayerVisibility` updates `activeLayers` without affecting `layers`.
- `appendStreamingDelta` accumulates text correctly and clears on `finalizeAssistantMessage`.

12.2 End-to-End Tests (Playwright)

`viewport.spec.ts`:

- Load the app with a mock WebSocket server.
- Send a pre-baked Step 1 bundle.
- Assert that the canvas is non-blank (pixel sampling).
- Send Step 2 bundle with `layer_mode = 'add'`.
- Assert that the canvas pixel content has changed.

`timeline.spec.ts`:

- Assert step markers render with correct count.
- Click step marker 3 → assert `stepIndex` store value equals 3.
- Click play → assert the play button changes to a pause icon.
- Press `ArrowRight` keyboard → assert `stepIndex` increments.

`hud.spec.ts` (visual regression):

- For each of 20 preset scenes (5 steps each): load, advance through all steps, take a screenshot at each step.
- Compare screenshots to golden images with a 0.5% pixel-difference threshold.

- Assert that the `ActiveEquationPanel` text matches the expected `hud_equation` LaTeX string for each step (read via `textContent` of the KaTeX output).
-

13. Extension Points

Adding a new transition type:

1. Add the new type to the `TransitionType` enum in `src/types/`.
2. Implement the animation in `TransitionController.ts` following the existing pattern.
3. No other changes required — `LayerManager` calls `TransitionController` generically.

Adding a new HUD panel element:

1. Add the component inside `HUDOverlay.tsx`.
2. Add any new state to `hudSlice.ts` and populate it from `setFromBundle`.
3. No Zustand store structural changes needed if the new state logically belongs to the HUD.

Supporting a new geometry type from the server:

1. Add handling in `BufferBuilder.ts` for the new `concept_type`.
2. Add the corresponding material in `MaterialFactory.ts`.
3. The `LayerManager` calls both generically — no changes needed there.

Dark / light theme: All colors are CSS custom properties on `:root`. A `data-theme` attribute on `<html>` switches between theme token sets. KaTeX output inherits text color from CSS. Three.js scene background and material colors read from CSS custom properties at scene init time via `getComputedStyle`.